오픈소스 반도체 설계 및 제작 종합설계 #5

FPGA 에뮬레이션 검증 킷 실습과 학습 내용 정리

2021-17385

산업공학과 김예원

[1] 학습 내용 요약 (1~4 주차)

1 주차에는 아두이노로 만든 탁구 게임 코드를 파이썬으로 다시 작성하여 게임의 동작 원리를 소프트웨어로 확인하였다. 2 주차에는 AI(Gemini)와의 대화만으로 만든 우주 침략자 게임의 베릴로그 RTL 을 Verilator 로 C++ 모델로 바꾸고, SystemC 테스트벤치와 SDL 화면 모델로 시뮬레이션하였다. RTL(Register Transfer Level)은 회로를 레지스터에 저장된 값이 클럭마다 조합 회로를 거쳐 다음 레지스터로 전달되는 과정으로 기술한 설계 수준이며, 베릴로그는 이를 작성하는 하드웨어 기술 언어이다.

3 주차에는 소프트웨어와 하드웨어의 차이를 게임 코드에서 확인하였다. 그래픽 장치를 다루는 방식이 VGA(좌표를 입력받아 색을 계산)에서 그래픽 LCD(내부 카운터로 화소를 순서대로 출력)로 바뀌었고, 같은 스프라이트도 아두이노는 순차 실행으로 그리지만 RTL 에서는 화소마다 비교 회로가 병렬로 동작한다. 또한 좌우 화살표의 이상 반응이 매 클럭 버튼을 확인하는 RTL 과 16.6ms 마다 입력을 주는 테스트벤치의 시간 척도 차이에서 생긴다는 것을 확인하였고, Q.13 에서는 화면 끝에서 한 번만 버튼을 확인하도록 고쳐 해결된 것을 확인하였다.

4 주차에는 FPGA 개발 도구(Quartus)와 Pi Pico 개발 환경을 설치하였다. Q.14 의 대화에서 AI 가 제시한 면적 평가를 실제 합성(yosys, ETRI 0.5μm 표준 셀)으로 검증한 결과, 원본은 AI 추정(12,000~15,000 게이트)보다 훨씬 작은 약 2,410 게이트였고 AI 의 재설계는 오히려 5.8% 컸다. Q.14 는 칩의 핀 수에 맞추기 위해 출력을 15 비트(좌표+화소)에서 3 비트(clk_o, v_sync, lcd_data)로 줄였고, 그 결과 FPGA 에서도 논리 소자가 24% 줄었다. 칩 제작은 RTL 작성 → 시뮬레이션 → 에뮬레이션 → 합성·배치·배선(ETRI050) → GDS 제출 → 제작 → 테스트의 순서로 진행되며, 같은 RTL 과 같은 테스트벤치가 각 단계에서 다른 도구를 거친다.

[2] 피드백 보완

수업에서 'v_sync 는 한 화면이 지나갔는지 확인하는 신호이고, 캐넌은 외계인보다 빨라야 한다'는 설명을 듣고 Q.14 코드에서 실제로 그런지 확인하였다. 이 게임의 화면은 가로 128 칸, 세로

64 칸이어서 한 화면을 그리는 데 128×64 = 8,192 클럭이 걸린다. 캐넌은 한 화면이 끝날 때(v_sync)마다 버튼을 확인해 한 칸 움직이고, 외계인은 네 화면(32,768 클럭)마다 한 칸 움직인다. 게임을 실행해 40 화면 동안 세어 보니 캐넌은 40 번, 외계인은 10 번 움직여 캐넌이 4 배 빨랐다. 그런데 플레이어가 쏜 총알도 외계인처럼 네 화면마다 한 칸씩 움직여서, 총알이 캐넌보다 느리다는 것도 알게 되었다.

코드에 나오는 숫자가 어디서 왔는지도 확인하였다. 이전 버전(Q.11)의 833,333 은 50MHz 클럭으로 1 초에 60 번 화면을 바꾸기 위한 값(50,000,000 ÷ 60)이었다. 외계인 25 마리는 위치를 하나하나 저장하지 않고, 무리 전체의 시작 위치 하나와 각 외계인이 살아 있는지를 나타내는 25 개의 비트만 저장한다. 외계인이 항상 함께 움직이기 때문에 가능한 방법이며, 25 마리의 위치를 모두 저장하는 것보다 저장 공간이 크게 줄어든다. 또 디자인 킷의 칩 핀 배치 파일을 보니 이 게임 칩은 28 핀이고 그중 16 개가 전원과 접지용이어서, 신호에 쓸 수 있는 핀은 12 개뿐이었다. Q.14 가 좌표 출력을 없애 입출력을 8 개로 줄인 이유를 여기서 확인할 수 있었다.

마지막으로 'always 블록 안에 너무 많은 내용을 넣지 말고 나누라'는 것이 중요하다는 피드백에 따라 Q.14 코드를 다시 보았다. 게임의 거의 모든 동작이 클럭에 맞춰 실행되는 always 블록 하나(약 150 줄)에 들어 있었고, 그 안에 클럭마다 값을 저장하는 문장(<=)과 바로 계산하는 문장(=)이 섞여 있었다. 문법 검사 도구인 Verilator 도 이 부분에 경고를 냈다. 값을 계산하는 부분과 저장하는 부분을 서로 다른 블록으로 나누어야 오류가 생기지 않는다는 것을 코드에서 확인하였다.

[3] FPGA 에뮬레이션 검증 킷 실습

지급받은 킷은 Cyclone IV E(EP4CE6) FPGA 보드(PSCE-TRANS)와 Raspberry Pi Pico 보드(PSCE-MI)로 구성된다. 두 보드는 2×6 핀 커넥터 세 개(Pico 보드의 J1~J3, FPGA 보드의 JB1, JC1, JD1)로 결합되고, USB 케이블 두 개를 PC 에 연결한다. 하나는 Pico(PC 와의 통신, 펌웨어 업로드), 다른 하나는 FPGA 의 JTAG 케이블(FT232H, 회로 설정)이다. WSL 에서 USB 장치를 쓰기 위해 Windows 의 usbipd 로 두 장치를 WSL 에 연결(bind, attach)하였다. 처음 전원을 넣었을 때 OLED 에는 Pico 에 미리 설치된 에뮬레이션 펌웨어의 시작 화면이 표시되었다(그림 1).

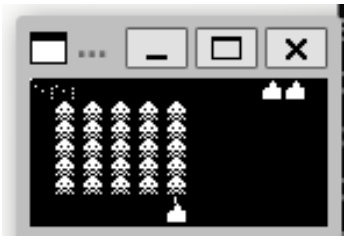


그림 1. 처음 전원을 넣었을 때의 화면 (PSCE-MI 펌웨어 시작 화면, Pico 의 초록 LED 와 FPGA 보드의 빨간 LED 점등)

(1) 에뮬레이션

Q.13 의 emulation 디렉토리에서 세 단계를 실행하였다. FPGA 설정(make config_trans)은 처음에 'Unknown device with IDCODE: 0x020f10dd' 오류로 실패하였다. JTAG 케이블은 FPGA 의 ID(Altera Cyclone IV E)를 정상적으로 읽었으나, apt 로 설치한 openFPGALoader 0.12.0 의 장치 목록에 이 ID 가 없었다. 최신 소스(v1.2.0)를 빌드하여 설치하자 'Load SRAM 100%, Done'으로 설정이 완료되었다. Pico 펌웨어 업로드(make upload_mi)는 'No drive to deploy' 오류로 실패하였다. 업로드 과정에서 Pico 가 펌웨어 수신 모드로 재시작하며 새 USB 저장 장치(RPI-RP2)가 되는데, 이 장치가 WSL 이 아닌 Windows 에 D: 드라이브로 연결되었기 때문이다. 빌드된 uf2 파일을 Windows 에서 D: 드라이브로 복사하여 업로드하였다.

이후 PC 테스트벤치(make build, make run)를 실행하자 에뮬레이터와 연결된 뒤 게임 화면이 그려졌다. 화면 모델의 창은 처음에 128×64 화소 크기로 열렸고(그림 2), 창 크기를 늘리면 같은 비율로 확대되었다(그림 3). 화면은 simulation 과 같은 SystemC 테스트벤치와 화면 모델로 그려지며, DUT 만 Verilator 모델 대신 FPGA 에서 동작한다. 다만 진행이 매우 느려, 외계인 두 마리를 격추하기까지 수 분이 걸렸다. 에뮬레이션(CA 모드)은 클럭 한 번마다 PC 가 USB 로 Pico 를 거쳐 FPGA 에 입력을 주고 출력을 읽어 오므로, FPGA 회로 자체의 속도(최대 약 70MHz)가 아니라 USB 왕복 시간이 속도를 정한다. 한 화면이 8,192 클럭이므로 한 번 왕복에 1ms 만 걸려도 한 화면에 약 8 초가 걸린다.



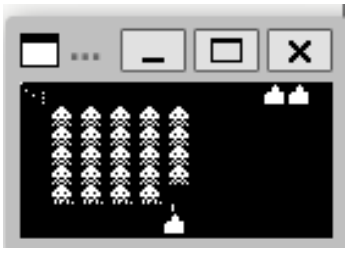


그림 2. 에뮬레이션 실행 직후의 기본 창 (128×64 화소). 화소를 받는 대로 한 점씩 그려 화면이 천천히 바뀜

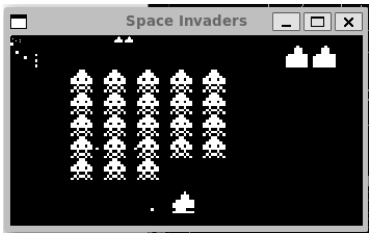


그림 3. 에뮬레이션 실행 화면, 창을 확대한 상태 (FPGA 에서 동작하는 Q.13 RTL 의 출력을 SystemC 테스트벤치가 그림)

(2) 프로토타이핑

application 디렉토리에서 테스트벤치 없이 게임 RTL 만 FPGA 에 올리고(make build, make config), Pico 에 프로토타이핑 펌웨어를 올렸다. Quartus 컴파일은 성공하였으나 'Timing requirements not met' 경고가 나왔다. 타이밍 제약 파일(SDC)이 없어 Quartus 가 클럭을 기본값 1GHz 로 가정하고 검사한 결과이며, 실제 클럭은 100kHz 이므로 동작에 문제가 없었다. 이 펌웨어는 Pico 의 PWM 으로 100kHz 클럭을 만들어 FPGA 에 공급하고, FPGA 의 clk_o 와 v_sync 에 인터럽트를 걸어 화소(lcd_data)를 받아 OLED 에 그린다. PC 의 키보드 입력 프로그램(make run)은 키 입력만 Pico 로 보낸다. 실행 결과 OLED 에 게임이 표시되었고, 키보드 입력에 따라 캐넌이 움직이고 총알이 발사되었다(그림 4). OLED 모듈이 보드에 90° 돌려 장착되어 있어 화면이 세로로 보인다.

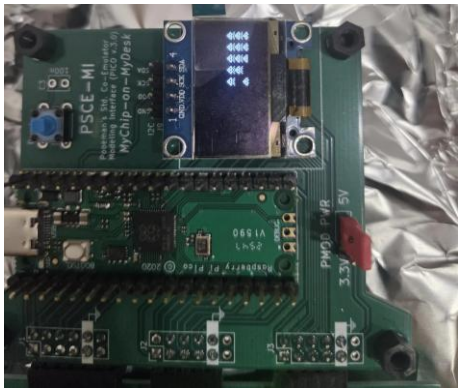


그림 4. 프로토타이핑 동작 화면 (Pico 가 공급하는 100kHz 클럭으로 FPGA 가 동작하고, 출력이 OLED 에 표시됨)

[4] 시뮬레이션, 에뮬레이션, 프로토타이핑 비교

같은 게임 RTL 을 세 가지 방식으로 실행한 결과를 표 1 에 정리하였다. 세 방식은 회로가 동작하는 장소와 클럭을 공급하는 주체가 다르다.

표 1. 세 가지 검증 방식 비교 (Q.13 우주 침략자 게임)

항목	시뮬레이션	에뮬레이션 (CA)	프로토타이핑
회로가 동작하는 곳	PC (Verilator C++ 모델)	FPGA	FPGA
클럭 공급	SystemC 테스트벤치	PC 가 USB 로 한 클럭씩	Pico PWM (100kHz)
화면 출력	PC (SDL)	PC (SDL, 같은 테스트벤치)	보드 OLED
관찰한 속도	약 45~53 프레임/초	한 화면에 수 초	약 12 프레임/초 (계산값)
내부 신호 관찰	모든 신호 (파형)	입출력만	입출력만

시뮬레이션은 모든 내부 신호를 볼 수 있지만 PC 의 계산 속도에 묶인다. 에뮬레이션은 회로가 실제 하드웨어에서 동작하고 시뮬레이션과 같은 테스트벤치로 결과를 비교할 수 있지만, 클럭마다 일어나는 통신 때문에 이번 실습에서는 시뮬레이션보다 느렸다. 상용 에뮬레이터가 여러 클럭 분량을 묶어 주고받는 방식을 쓰는 이유가 이 병목에 있다. 프로토타이핑은 FPGA 가 실제 클럭으로 스스로 동작하여 가장 빠르지만, PC 에서 내부 동작을 관찰하기 어렵다. 칩은 제작 후 고칠 수 없으므로, 세 방식을 차례로 거치며 각 단계에서 확인할 수 있는 것을 확인한 뒤 제작으로 넘어간다는 것을 확실히 알 수 있었다.